Learn Classes by Building

Beginner tasks with clear situations, small steps, and examples to try.

What you will practice
Classes and objects • __init__ • attributes • self • methods • changing an object • lists of objects

How to use this workbook
Do the tasks in order. Type and run your code after each small step. If you get stuck, use the hint and your class notes.

A tiny reminder

Class: a reusable design. **Object:** one thing made from that design. **Attribute:** data an object remembers. **Method:** an action written inside a class. self means the particular object using the method.

Example shape (you will fill in the details in the tasks):

```
class Thing:
    def __init__(self, detail):
        self.detail = detail

item = Thing("example")
```

Start here

1. Make a class for a pet

Situation: A pet shop wants to keep track of each pet's name. Every pet is the same kind of record, but each has its own name.

Your task:

1. Create a class named Pet.
2. Add __init__ so a new Pet receives a name and saves it as self.name.
3. Create two pets with different names.
4. Print each pet's name.

Hint: Use Pet("Milo") to create one object.

Check your work: You should see both names, and they should be different.

_____

_____

2. Add more details to each object

Situation: A teacher needs a simple record for each student: name, age, and favorite subject.

Your task:

1. Create a class named Student.
2. Make __init__ receive name, age, and favorite subject.
3. Save all three values on the object using self.
4. Create two students and print their details with labels.

Hint: A line such as self.age = age stores the age on this student.

Check your work: Each student's details stay grouped together, and the two records show different information.

_____

_____

3. Give an object an action

Situation: A classroom display has a welcome sign for each student. The message should use the name stored in that student object.

Your task:

1. Use the Student class from Task 2.
2. Add a method named say_hello(self).
3. Have the method print a short greeting using self.name.
4. Call the method once for each student.

Hint: Call it with student_one.say_hello(); Python supplies self for you.

Check your work: Each greeting uses the correct student's name.

_____

_____

Practice with actions and object data

4. Calculate something with a method

Situation: A small garden shop sells rectangular plant beds. The shop wants to know how much ground each bed covers.

Your task:

1. Create a class named GardenBed with length and width attributes.
2. Add an area(self) method that returns length multiplied by width.
3. Create a bed that is 4 units long and 3 units wide.
4. Print the value returned by area().

Hint: A method can return an answer with return; print the result where you call the method.

Check your work: The example bed's area is 12.

_____

_____

5. Change one object using a method

Situation: A game player starts with 10 points. When the player collects a star, their score should increase by 1.

Your task:

1. Create a class named Player with a name and a score that starts at 0.
2. Add a method named collect_star(self) that adds 1 to this player's score.
3. Create two players.
4. Let the first player collect two stars and the second collect one. Print both scores.

Hint: Use self.score += 1 inside the method.

Check your work: The first player has 2 points and the second has 1. One player's actions do not change the other player's score.

_____

_____

6. Model a library book

Situation: A library needs to know whether each book is on the shelf. A book can be borrowed only when it is available.

Your task:

1. Create a class named LibraryBook with a title and an availability value that starts as True.

2. Add a borrow(self) method. If available, change availability to False and print a borrowed message.

3. If it is already unavailable, print a message saying it has already been borrowed.

4. Create one book and call borrow() twice. Print its availability after the calls.

Hint: Use an if/else statement and update self.is_available.

Check your work: The first call borrows the book. The second call explains it is already borrowed. Availability ends as False.

_____

_____

Put objects together

7. Keep a class of students in a list

Situation: The teacher from Task 2 now wants to view the whole class instead of printing students one at a time.

Your task:

1. Create at least three Student objects.
2. Put the objects into a list called students.
3. Use a for loop to print the name and favorite subject of each student.
4. Add the ages and calculate the class's average age.

Hint: Inside the loop, the variable (for example student) refers to one Student object.

Check your work: Every student appears once, and the average age is calculated from all students in the list.

_____

_____

8. Add a simple result to a student

Situation: A teacher wants each student record to tell whether the student passed. A mark of 40 or more is a pass.

Your task:

1. Add a marks attribute to the Student class.
2. Add a method named has_passed(self) that returns True when marks are at least 40 and False otherwise.
3. Create one student with 65 marks and one with 32 marks.
4. Print each name and the value returned by has_passed().

Hint: Use return self.marks >= 40.

Check your work: The student with 65 gets True; the student with 32 gets False.

_____

_____

9. Small project: class library shelf

Situation: A library volunteer wants a tiny program that lists books and lets one book be borrowed. This combines a class, objects, a method, and a list.

Your task:

1. Use your LibraryBook class from Task 6. Create three different books.
2. Store the three objects in a list called books.
3. Loop over the list and print each book's title and whether it is available.
4. Borrow one chosen book, then loop again to show the updated status of every book.

Hint: Call borrow() on the selected object from the list.

Check your work: Only the chosen book changes to unavailable; the other books remain available.

_____

_____

Ready to use classes?

You are ready for beginner class programs when you can create a class, make more than one object, store each object's own data, call a method, and use a list when you have a group of objects. If one part feels confusing, repeat that task with a different everyday example.

Reflection: Which task helped you understand self best? What new class would you like to create?

_____

_____

NEXT STEP: MID-LEVEL PRACTICE

These tasks combine ideas from the beginner section. Try them after you can create objects, use attributes, and call simple methods.

Keep in mind: Make one small change at a time, run your program, and use the check at the end to see whether the behavior is right.

Different concepts, new situations

10. Track a small shop's stock

Situation: A stationery shop has several products. Each product needs its own name and quantity. The shopkeeper wants to know which products need restocking.

Build it:

1. Create a Product class with name and quantity attributes.
2. Create at least four products and put them in a list.
3. Add a method needs_restock(self) that returns True when quantity is below 5.
4. Loop over the products and print the names of the ones that need restocking.

Hint: The loop variable is one Product object. Call its method and use an if statement.

Check: Only products with fewer than 5 items are listed.

11. Work out a student's result

Situation: A teacher records marks in three subjects for each student. Each student should be able to calculate their own average and decide whether they passed.

Build it:

1. Create a Student class with a name and a list of three marks.
2. Add an average_marks(self) method that returns the average.
3. Add a has_passed(self) method that returns True when the average is 40 or more.
4. Create two students with different marks. Print each name, average, and pass result.

Hint: The average is sum(self.marks) / len(self.marks).

Check: The methods use the marks belonging to the student who calls them; test one student who passes and one who does not.

12. Make a simple shopping cart

Situation: A customer adds priced items to a cart. The cart should remember its items and calculate the total when asked.

Build it:

1. Create a ShoppingCart class with an empty list attribute called prices.
2. Add add_item(self, price) to put a price in the list.
3. Add get_total(self) that returns the sum of the prices.
4. Create two carts, add different items, and print both totals.

Hint: Start with self.prices = [] in __init__.

Check: The totals match the prices added to each cart, and adding an item to one cart does not change the other.

Mid-level practice

13. Manage a bank account safely

Situation: A person wants a small program to keep their account balance correct when they deposit or withdraw money.

Build it:

1. Create a BankAccount class with owner and starting balance.

2. Add deposit(amount) that adds a positive amount and returns True; invalid amounts return False.

3. Add withdraw(amount) that subtracts the amount only when it is positive and not more than the current balance.

4. Create two accounts. Try a valid deposit, a valid withdrawal, and a withdrawal that is too large. Print the final balances.

Hint: Check the amount and available balance before changing self.balance.

Check: A failed action leaves the balance unchanged. Each account keeps its own balance.

14. Find a book in a library list

Situation: A librarian has a list of books and wants to find one by title. The program should say whether it is available.

Build it:

1. Create a LibraryBook class with title and availability attributes.

2. Create at least three books and store them in a list.

3. Write a function or a short loop that looks for a title entered in a variable such as search_title.

4. When the title is found, print its availability. If no title matches, print “Book not found.”

Hint: Compare book.title with the title you are searching for.

Check: Searching for a title in the list gives the correct status; searching for a missing title gives the not-found message.

15. Build a simple movie watchlist

Situation: A student keeps a watchlist. Each movie has a title and rating, and the student wants to view only movies rated 8 or higher.

Build it:

1. Create a Movie class with title and rating.

2. Add a method is_recommended(self) that returns True when the rating is at least 8.

3. Create four movie objects and store them in a list.

4. Loop through the list and display the titles of recommended movies.

Hint: Call movie.is_recommended() for each item in the list.

Check: Only movies rated 8 or higher are displayed.

A good next step

Choose one task and add one small improvement of your own. For example, add another method, another object, or a useful message for an invalid action. Keep the class responsible for behavior that belongs to the object.